

Low-Bit Data Processing Using Multiple-Output Spiking Neurons with Non-linear Reset Feedback

Sanja Karilanova, Subhrakanti Dey, Ayça Özçelikkale

Department of Electrical Engineering, Uppsala University, Sweden
{Sanja.Karilanova, Subhrakanti.Dey, Ayca.Ozcelikkale}@angstrom.uu.se

Abstract—Neuromorphic computing is an emerging technology enabling low-latency and energy-efficient signal processing. A key algorithmic tool in neuromorphic computing is spiking neural networks (SNNs). SNNs are biologically inspired neural networks which utilize stateful neurons, and provide low-bit data processing by encoding and decoding information using spikes. Similar to SNNs, deep state-space models (SSMs) utilize stateful building blocks. However, deep SSMs, which recently achieved competitive performance in various temporal modeling tasks, are typically designed with high-precision activation functions and no reset mechanisms. To bridge the gains offered by SNNs and the recent deep SSM models, we propose a novel multiple-output spiking neuron model that combines a linear, general SSM state transition with a non-linear feedback mechanism through reset. Compared to the existing neuron models for SNNs, our proposed model clearly conceptualizes the differences between the spiking function, the reset condition and the reset action. The experimental results on various tasks, i.e., a keyword spotting task, an event-based vision task and a sequential pattern recognition task, show that our proposed model achieves performance comparable to existing benchmarks in the SNN literature. Our results illustrate how the proposed reset mechanism can overcome instability and enable learning even when the linear part of neuron dynamics is unstable, allowing us to go beyond the strictly enforced stability of linear dynamics in recent deep SSM models.

Index Terms—event-based signal processing, spiking neural networks (SNNs), state-space models (SSMs), reset-mechanism, neuromorphic.

I. INTRODUCTION

Signal processing and machine learning methods are typically implemented in the conventional *von Neumann* computing architectures. The effectiveness of these methods often comes at the cost of high energy consumption for computation [1]. Additionally, several layers of data processing are desired in an increasing number of low-power internet of things (IoT) edge applications, such as large scale deployments for remote monitoring of industrial sites, and monitoring/surveillance with battery-powered drones. Neuromorphic computing, where insights from neuroscience are utilized for building computing architectures and processing data, has recently emerged as a promising solution for energy-efficient data processing [2]–[4]. Accordingly, data processing on neuromorphic chips has

shown to achieve significant gains in terms of low-latency and energy-efficiency compared to solutions implemented on the conventional chips for a range of applications, including constraint satisfaction problems and least absolute shrinkage and selection operator (LASSO) problems [3].

A key concept in neuromorphic computing is the encoding and decoding of information using spikes, i.e. impulses. The typical encoding scheme is binary spikes where in discrete-time data processing, the temporal information is represented as a sequence of 0's and 1's. In other words, the information is quantized with two levels, i.e. with 1-bit resolution. Representation of information with 1-bit resolution allows conversion of multiplication operations to additions, resulting in lower number of arithmetic operations and power consumption [4].

The typical computational model used in neuromorphic hardware is spiking neural networks (SNNs) [3], where the basic unit of computation is a spiking neuron. In a standard artificial neural network, the input to a neuron is a continuous-valued variable and the neuron apply a static transformation to its input. In contrast, the biological neurons have a state, such as a membrane voltage, and they process spike-trains inputs and produce spike trains outputs. Spiking neurons in SNNs mimic these behavior. One of the most popular models is the Integrate and Fire (IF) model which integrates its input, and fires a spike and resets its state to its neutral value whenever it fires a spike [5].

Neuron models in SNNs are typically designed using biological plausibility based on neuroscience models [5]–[8]. Here, we combine insights from state space models (SSMs) in signal processing and machine learning, with insights from neuroscience to develop new neuron models to use in neural networks (NNs) and illustrate their performance. SSMs have been used in various successful signal processing methods, lately they have even obtained comparable performance to transformers in deep NN literature [9]–[11]. However, the typical SSMs in NNs do not use low-bit processing. In particular, in contrast to SNNs, where the output of the neurons are represented using spikes which are sparse in time and has limited amplitude resolution, the outputs of SSMs in a typical deep SSM are continuous-valued. Motivated by the computational complexity and energy gains that can be obtained using low-bit data processing as well as promising energy efficiency results in neuromorphic computing [2]–[4], in this article we propose SSM based NN models with low-bit activation and hence communication between neurons.

A central question in our investigation is "How should we

S. Karilanova and A. Özçelikkale acknowledge the support of Center for Interdisciplinary Mathematics (CIM), and AI4Research, both in Uppsala University.

The computations were enabled by resources provided by the National Academic Infrastructure for Supercomputing in Sweden (NAISS), partially funded by the Swedish Research Council through grant agreement no. 2022-06725.

model low-bit SSM-based NNs?" Linear SSMs cannot model the nonlinear behavior existing in various signals, while non-linear SSMs are difficult to develop, interpret and use in data processing pipelines. Hence, we propose a SSM that combines a linear state-transition dynamics with a non-linear feedback through reset.

A. Contributions

Our contributions can be summarized as follows:

- We propose a multiple-output stateful neuron model with a non-linear feedback mechanism through a state reset. In contrast to the traditional neuron models in SNN literature, this model allows a more general model structure through a general state transition matrix, multiple-outputs for the neuron and a general reset mechanism with learnable parameters. In contrast to typical SSM based models in NN literature, this neuron model brings low-bit information representation with spikes, non-linear feedback and allows instability in the SSM through the state transition matrix.
- Our development generalizes the reset mechanism in SNN literature, which is typically based on neuroscience models and focuses on particular state variables. Our development clearly conceptualizes the differences between the spiking function, the reset condition, the reset action.
- Using the neuromorphic computing benchmark dataset [4] from the Multilingual Spoken Word Corpus (MSWC) dataset [12], DVS-Gesture dataset [13] and sequential MNIST (sMNIST) dataset [14], we illustrate that performance at the same level as the benchmark results in the literature can be obtained using the proposed neuron model with non-linear feedback.
- Our results illustrate that in order to have satisfactory performance, one does not need to have stable linear SSM dynamics, hence one does not need to constrain the eigenvalues of the state transition matrix, which is a central concern in developing models in SSM-based NN literature. Our results provide a starting point for investigation of a richer family of SSM models in neural networks.

We note that the typical neuron model in SNN literature is single output, with only a few recent exceptions [15]. In contrast to the main-stream single output neuron model in the SNN literature, here we consider a neuron model with multiple outputs. Different from the linear model in [15] with no reset and stable linear dynamics, here we consider a non-linear feedback through a state reset mechanism with learnable parameters, and allow unstable state-transition matrices.

II. RELATED WORK

The Integrate-and-Fire (IF) neuron, one of the most basic neuron models for SNNs, is closely related to the Time Encoding Machines [16]. Time Encoding Machines (TEMs) are event-based sensors [16]. Integrate-and-Fire (IF) TEM fires a spike when the integral of a moving window of a signal exceeds a certain threshold [16]. This IF behavior of TEMs is similar to IF behaviour in neurons, where the main

difference is that TEMs are typically studied under continuous-amplitude inputs whereas IF neurons in SNNs typically use spike trains. Both IF TEMs and IF neurons produce spike trains as output. The operation of IF TEM can be directly associated with non-uniform sampling leading to sufficiency results for reconstruction of signals from TEM outputs [16]. Recently, researchers started to show growing interest in the TEM based event-based sampling framework, leading to a range of signal reconstruction results [17]–[19].

State-space models (SSMs) are heavily utilized in the fields of signal processing and control for system modeling and modeling of temporal dependency. In deep learning, popular models for capturing long-term dependencies in temporal data have been recurrent architectures, complex gated models and transformers among the few. In the last few years, SSM based deep neural networks have emerged as a promising simpler alternative by achieving competitive performance on a large set of long-range sequence modeling tasks, see S4 [9] and S5 [11]. The promise of such structures has also been shown under the neuromorphic low-bit resolution, event-based data [20], [21].

Recently, making SSM based NNs more computationally efficient has been studied using quantization of the weights [22], and quantization of combination of weights, SSM parameters, activation function under both quantization-aware training and post-training quantization [23], as well as under quantization aware fine-tuning [24]. Quantization of the activation function, or the output of SSMs to 1-bit, is closely related to the inherent and biologically-inspired 1-bit information processing in SNNs. Hence, even more recently, there has been an effort to bridge SSMs and SNNs [15], [25]–[27] [28]. However, drawing this bridge is not straightforward due to the linear nature of SSMs and recurrently non-linear nature of SNNs [29], especially due to the resetting of the membrane potential once a neuron has spiked. To the best of our knowledge, no prior work has investigated modeling a reset in deep-SSM especially in a one-bit and multiple output neuron setting. In addition, the works in deep SSMs have been restricted to stable initialization of SSMs in various ways [10] due to lack of mechanism that will stabilize the learning. In this paper we show that reset in deep SSMs can overcome unstable initialization and training.

III. SYSTEM FORMULATION

A. Preliminaries

1) *General Spiking Neuron Model:* A single general feed-forward discrete-time n -state dimensional spiking neuron can be defined as

$$\mathbf{v}[t+1] = \mathbf{A}\mathbf{v}[t] - \mathbf{R}s_{out}[t] + \mathbf{B}i[t] \quad (1a)$$

$$s_{out}[t] = f_{\Theta}(\mathbf{v}[t]) = \begin{cases} 1 & \text{if } \mathbf{v}[t] \in \Theta \\ 0 & \text{otherwise} \end{cases} \quad (1b)$$

where $i[t] \in \mathbb{R}^{1 \times 1}$ is the input to the neuron, $s_{out}[t] \in \mathbb{R}^{1 \times 1}$ is the output of the neuron and $\mathbf{v} \in \mathbb{R}^{n \times 1}$ is the state variable of the neuron. The matrices $\mathbf{A} \in \mathbb{R}^{n \times n}$, $\mathbf{B} \in \mathbb{R}^{n \times 1}$, $\mathbf{R} \in \mathbb{R}^{n \times 1}$ represents the leak, impact of the input and the reset feedback, respectively. The spiking behavior is described by $f_{\Theta}(\cdot)$, where the neuron spikes when the state enters into the region described by Θ . See Table I for the key notation used in this paper.

TABLE I: Table of notation

Notation	Meaning
n	Dimension of a neuron = Number of variables in the state
n_{out}	Output dimension = Number of output channels of a neuron
$\mathbf{v}[t] \in \mathbb{R}^n$	State of the neuron
$\mathbf{i}[t] \in \mathbb{R}^1$	Input of the neuron
$\mathbf{s}_{out}[t] \in \mathbb{R}^{n_{out}}$	Spike output of the neuron
$f(\cdot)$	Activation/Spiking function
$\mathcal{R}_c(\cdot)$	Reset condition parametrization
$\mathcal{R}_a(\cdot)$	Reset action
c_{in}	Number of input channels for the network
c_{out}	Number of output channels for the network
h	Number of neurons in a hidden layer

The above formulation considers a type of soft-reset mechanism where the output spikes $\mathbf{s}_{out}[t]$ are multiplied by $\mathbf{R}$ and subtracted from the state, see (1a). Another reset mechanism that is popular in the SNN literature is hard-reset where the neuron state is set to a predefined reset state $\mathbf{v}_{rst}$, e.g. to a state of all zeros, once spiking occurs [30]. Under such hard-reset, the $\mathbf{R}\mathbf{s}_{out}[t]$ term is removed from (1), and the below additional operation is performed:

$$\mathbf{v}[t+1] = \begin{cases} \mathbf{v}_{rst} & \text{if } s_{out}[t] = 1 \\ \mathbf{v}[t+1] & \text{otherwise} \end{cases} \quad (2)$$

Spiking neural networks are built by connecting neurons such that the output spikes $\mathbf{s}_{out}^L[t] \in \mathbb{R}^{L_{pre} \times 1}$ from the previous (pre-synaptic) layer serve as inputs to the next (post-synaptic) layer $\mathbf{I}[t] \in \mathbb{R}^{L_{post} \times 1}$, where L_{pre} and L_{post} is the number of neurons in the pre and post synaptic layer. Specifically, $\mathbf{I}[t] = \mathbf{W} \times \mathbf{s}_{out}^L[t]$ where $\mathbf{W} \in \mathbb{R}^{L_{post} \times L_{pre}}$ is the weight matrix representing synaptic connection strength between the two layers.

2) *Popular Spiking Neuron Models*: The discrete-time Leaky Integrate-and-Fire (LIF) neuron is defined as [5], [7]:

$$u[t+1] = \alpha u[t] - \alpha \theta s_{out}[t] + (1 - \alpha)i[t] \quad (3a)$$

$$s_{out}[t] = f_{\theta}(u[t]) = \begin{cases} 1 & \text{if } u[t] \geq \theta \\ 0 & \text{otherwise.} \end{cases} \quad (3b)$$

where $u[t]$ is the state variable representing the membrane potential. The LIF neuron is a special case of the generalized spiking neuron in (1). The LIF neuron is obtained by setting $n = 1$, $\mathbf{v} = u$, $\mathbf{A} = \alpha$, $\mathbf{B} = (1 - \alpha)$, $\mathbf{R} = \alpha\theta$, and $\Theta = \{u \geq \theta\}$ where $\theta = 1$ for a standard LIF neuron.

We now discuss the Adaptive Leaky Integrate-and-Fire (adLIF) neuron, which is an extension of the LIF neuron with an additional recovery state variable, $v[t]$ [7]. Discrete-time adLIF neuron is defined as follows

$$u[t+1] = \alpha u[t] - \alpha \theta s_{out}[t] + (1 - \alpha)i[t] - (1 - \alpha)v[t] \quad (4a)$$

$$v[t+1] = \alpha u[t] + \beta v[t] + b s_{out}[t] \quad (4b)$$

$$s_{out}[t] = f_{\theta}(u[t]) \quad (4c)$$

The adLIF neuron is a special case of the generalized spiking neuron in (1). It is obtained by setting $n = 2$ and

$$\mathbf{v} = \begin{bmatrix} u \\ v \end{bmatrix}, \mathbf{A} = \begin{bmatrix} \alpha & -(1 - \alpha) \\ a & \beta \end{bmatrix}, \mathbf{B} = \begin{bmatrix} 1 - \alpha \\ 0 \end{bmatrix}, \mathbf{R} = \begin{bmatrix} -\alpha\theta \\ b \end{bmatrix}.$$

Remark 1. Although in Section III-A1 we provide a general spiking neuron model with arbitrary number of n state

variables, we emphasize that the existing popular SNN models typically have a state dimension of $n = 1$ (LIF) or $n = 2$ (example adLIF) as illustrated in the above. Other examples of popular but less used spiking neuron models include Izhikevich [31] with $n = 2$, and Hodgkin-Huxley [32] with $n = 4$. Recently, motivated by the developments in SSM models in neural networks, a limited number of works [15], [25], [26], have investigated spiking neuron architectures with neurons with higher state dimensions.

3) *State Space Models*: A discrete-time time-invariant linear SSM can be written as [33]:

$$\mathbf{v}[t+1] = \mathbf{A}_s \mathbf{v}[t] + \mathbf{B}_s \mathbf{I}[t] \quad (5a)$$

$$\mathbf{y}[t] = \mathbf{C}_s \mathbf{v}[t] + \mathbf{D}_s \mathbf{I}[t], \quad (5b)$$

where $\mathbf{v}[t]$, $\mathbf{I}[t]$, $\mathbf{y}[t]$ denote the state vector, input vector and output vector, respectively. Here, the state transition matrix $\mathbf{A}_s \in \mathbb{R}^{n \times n}$ describes the internal recurrent behavior of SSM. The matrices $\mathbf{B}_s \in \mathbb{R}^{n \times n_{in}}$, $\mathbf{C}_s \in \mathbb{R}^{n_{out} \times n}$, $\mathbf{D}_s \in \mathbb{R}^{n_{out} \times n_{in}}$ describe the interaction of the SSM through its inputs and outputs. By stacking layers that consist of a linear SSM, i.e., (5), and element-wise nonlinearity applied to the output $\mathbf{y}$ of the linear SSM, SSM networks are formed [9]–[11].

B. Comparison of Spiking Neuron Models and Linear SSMs

We now compare the general spiking neuron model in Section III-A1 and linear SSM in Section III-A3. The leak matrix $\mathbf{A}$ in (1a) and the state transition matrix $\mathbf{A}_s$ in (5a) drive the linear part of the state transition in both models. Similarly, both $\mathbf{B}$ and $\mathbf{B}_s$ parameterize the impact of the input. However, one difference is how the output is obtained. In the spiking neuron the output is obtained by applying a non-linear spiking function on the raw state variable $\mathbf{v}[t]$, see (1b). In the linear SSM, the output is obtained by linear transformation of the state variable $\mathbf{v}[t]$, see (5b). Alternatively, in a deep SSM, the output of the layer is obtained using a real, continuous-valued activation function on $\mathbf{y}[t]$ of (5b) [9]–[11].

Another major difference of the models in Section III-A1 and Section III-A3 is the lack of reset mechanism in the model of Section III-A3. This is a key difference, which we focus on in this paper. The reset mechanism in the spiking neuron, as defined in (1a) or (2), enforces state discontinuities, i.e. reset, as a response to spikes in its output. This type of non-linear system feedback is not present in the linear SSM model in (5). We note that the element-wise non-linearity applied in deep SSM are only used to change the output of the layer and is not provided as a feedback to a layer; hence does not serve a reset mechanism.

IV. PROPOSED MULTIPLE-OUTPUT NEURON MODEL WITH RESET

A. Motivation

We now discuss several aspects of our motivation behind the proposed neuron model.

1) *1-bit Spike-based Information Processing*: Neural networks operating under low-bit precision communication constraints must efficiently transmit information between neurons

in the neural network. The spiking mechanisms in SNNs inherently adhere to this low-bit information processing paradigm, enabling energy-efficient information transfer. However, the performance of deep-SSM under such constraints remains under-explored, especially under the extreme 1-bit constraint for the communication between neurons.

2) *Neuroscience Perspective on Reset*: Biological neurons exhibit resetting dynamics after spiking, which plays a crucial role in their encoding of temporal information. A fundamental open question is how to formally incorporate reset mechanisms into SSM-based neural architectures and whether such mechanisms can enhance expressivity under 1-bit processing of temporal information.

3) *Beyond Enforced Stability*: SSM-based networks typically enforce a strictly stable dynamics both in the initialization as well as throughout training [10]. While this ensures well-behaved optimization in long-sequence operations, it limits the range of dynamical behaviors that can be represented. For example, strict stability conditions in the state transition may contribute to dead neuron problem; and fast decay of activity that has been initiated in the network due to specific temporal patterns, limiting the ability to capture long-range dependencies. In this paper, we allow unstable dynamics and investigate whether reset mechanisms can prevent divergence in such models, as well as the associated performance trade-offs.

4) *Beyond Linearity*: SSM-based neurons are typically modeled as linear systems. However, insights from signal processing, control theory, and neuroscience suggest that nonlinear dynamics could be fundamental to biological and engineered systems. Introducing reset mechanism to linear SSM-based neurons provides a natural extension toward nonlinearity, offering a stepping stone to more expressive and biologically plausible SSM-based architectures.

B. Proposed Model

Our proposed multiple-output SSM-based neuron with general reset mechanism is defined as:

$$\mathbf{v}[t+1] = \mathbf{A}\mathbf{v}[t] + \mathbf{B}i[t] \quad (6a)$$

$$\mathbf{y}[t] = \mathbf{C}\mathbf{v}[t] + \mathbf{c}_{bias} \quad (6b)$$

$$\mathbf{s}_{out}[t] = f_{\theta}(\mathbf{y}[t]) \quad (6c)$$

$$\mathbf{v}[t+1] = \begin{cases} \mathcal{R}_a(\mathbf{v}[t+1]) & \text{if } \mathcal{R}_c(\mathbf{y}[t]) \in \Theta \\ \mathbf{v}[t+1] & \text{otherwise} \end{cases} \quad (6d)$$

where $i[t] \in \mathbb{R}^{1 \times 1}$ is the input to the neuron at time step t and $\mathbf{s}_{out}[t] \in \mathbb{R}^{n_{out} \times 1}$ is the spiking output of the neuron at time step t produced by the spiking function $f_{\theta}(\cdot)$. The internal linear dynamics is parametrized by the matrix parameters $\mathbf{A} \in \mathbb{C}^{n \times n}$, $\mathbf{B} \in \mathbb{R}^{n \times 1}$ whereas $\mathbf{C} \in \mathbb{C}^{n_{out} \times n}$, and $\mathbf{c}_{bias} \in \mathbb{C}^{n_{out} \times 1}$ contributes to the output. Here, motivated by deep SSM literature, we consider a possibly complex-valued state dynamics through complex $\mathbf{A}$ whereas the input and output are real-valued through real-valued $\mathbf{B}$ and $f_{\theta}(\cdot)$, respectively.

Here, Θ defines a region in the range space of $\mathcal{R}_c(\cdot)$. At each time step, if the reset condition, $\mathcal{R}_c(\mathbf{y}[t]) \in \Theta$ based on the output $\mathbf{y}$, is satisfied then the reset action, $\mathbf{v}[t+1] =$

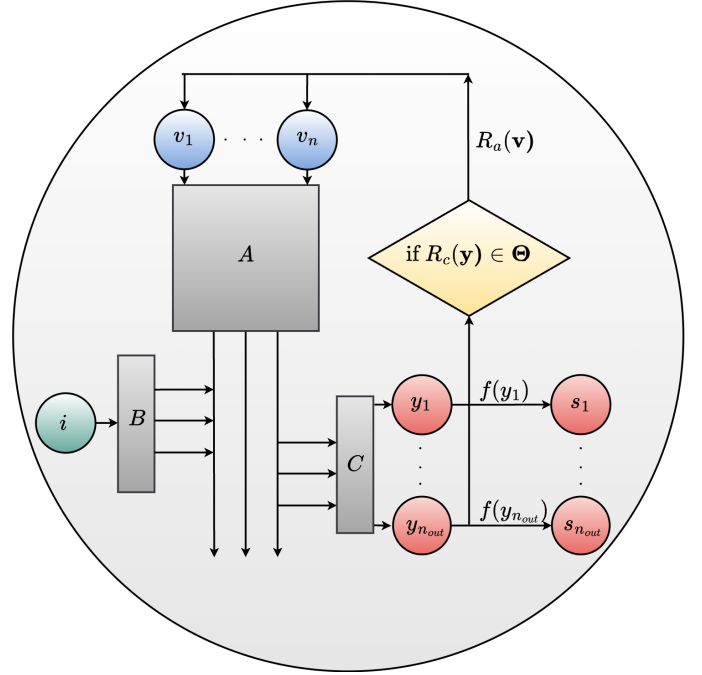


Fig. 1: Illustration of dynamics of a neuron with multiple-outputs and a general reset mechanism.

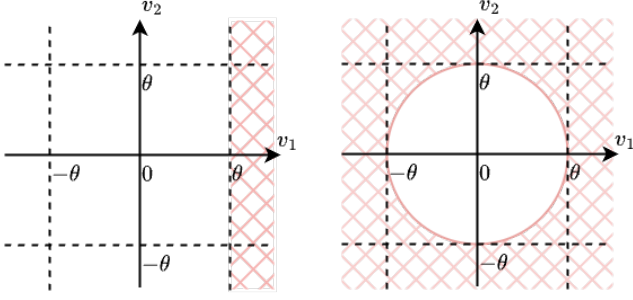
$\mathcal{R}_a(\mathbf{v}[t+1])$, is applied to the state. See Figure 1 for an illustration.

1) *Spiking function*: In this paper, as the spiking function, $f_{\theta}(\cdot)$, we use the most common spiking function – the Heaviside step function with threshold 1. However, in most of the existing works in SNN literature, the spiking function is usually applied to the state variables and furthermore, it is typically applied only to one of the state variables which corresponds to the membrane potential. Here, we make a distinction between the state variable $\mathbf{v}$ and the output of the neuron $\mathbf{y}$, and apply the spiking function to the output $\mathbf{y}$ of the neuron rather than $\mathbf{v}$. In particular, we apply $f_{\theta}(\cdot)$ element-wise to $\mathbf{y}$ such that for $i = 1, \dots, n_{out}$ we have

$$f_{\theta}(y_k[t]) = \begin{cases} 1 & \text{if } \Re(y_k[t]) + \Im(y_k[t]) > 1 \\ 0 & \text{otherwise.} \end{cases} \quad (7)$$

This formulation allows the neuron to produce n_{out} outputs regardless of the state dimension.

At first glance, (7) seems to impose a spiking threshold of 1 for all output channels of the neuron. We recall that the model includes trainable $\mathbf{C}$ and $\mathbf{c}_{bias}$, hence different outputs can in fact have different spiking thresholds. This feature allows diversity in the output and may be used to compensate for the negative effect of the 1-bit amplitude resolution in each output channel of the neuron. In particular, a multiple output neuron with different learnable spiking thresholds for each channel can be semantically interpreted as a single-output neuron with multiple thresholds and spike levels, e.g. a neuron that can output graded spike events as in Loihi 2 [3], [24]. Hence, the multiple-outputs of (6) may be interpreted effectively as a projection of the amplitude resolution of a graded spike into space, i.e., onto different channels in the output, with the added benefit of using different, possible learnable parts of the output space for different channels through learnable $\mathbf{C}$. The



(a) adLIF neuron reset condition with $v_1 > \theta$. (b) Proposed neuron reset condition for the case of $\|v\|_2 \geq \theta$.

Fig. 2: Visualization of the reset condition region, represented by red cross hatch, for $n = 2$ dimensional neurons.

positive effect of increasing n_{out} , and hence possible diversity, is illustrated in our numerical results in Section VI-H.

2) *Reset Condition*: In biologically inspired computational neuron models, the reset condition $\mathcal{R}_c(\cdot)$ is typically linked to the spiking function, meaning that a reset occurs whenever the neuron emits a spike. However, in this work, we introduce a key distinction: the spiking and reset mechanisms operate independently, allowing for separate dynamics.

We further differentiate between hard and soft reset conditions. Instead of solely indicating whether a reset should occur, a soft reset can provide quantitative information about the reset magnitude. However, in this work (6d), we employ a hard reset condition, where the only information transmitted is a binary decision on whether to reset or not.

In this paper, we consider a hard reset condition but decoupled from the spiking mechanism. Specifically, the reset function $\mathcal{R}_c(\cdot)$ is defined using the Heaviside step function, combined with the Euclidean norm and a learnable bias term $r_{bias} \in \mathbb{R}^{1 \times 1}$ as follows:

$$\mathcal{R}_c(\mathbf{y}[t]) \in \Theta \equiv \frac{1}{n_{out}} \|\mathbf{y}[t]\|_2 + r_{bias} \geq 1 \quad (8)$$

Note, in an adLIF neurons with state variables $\mathbf{v} = [v_1, v_2]^T \in \mathbb{R}^{2 \times 1}$, the reset condition is triggered when the membrane potential exceeds a threshold i.e., $v_1 \geq \theta$, see the reset condition region in Figure 2a. On the other hand, the proposed reset condition is possibly based on all state variables through $\mathbf{y}$. Consider the following example: Let $\mathbf{v} \in \mathbb{R}^{2 \times 1}$, $\mathbf{C} = 2\mathbf{I} \in \mathbb{R}^{2 \times 2}$, $\mathbf{c}_{bias} = 0$, $r_{bias} = 1 - \theta$. Then the reset condition becomes $\mathcal{R}_c(\mathbf{y}) = \|\mathbf{v}\|_2 \geq \theta$, see Figure 2b for an illustration.

Remark 2. In most commonly used spiking neuron models in SNNs, the reset condition coincides with spike generation, i.e., the reset condition is met when the neuron emits a spike. We generalize this by decoupling the two processes, using distinct functions, $f_\theta(\cdot)$ for spike generation and $\mathcal{R}_c(\cdot)$ for the reset condition. Furthermore, in most neuron models used in SNNs, the spiking depends only on one of the state variables; and hence also the reset condition. In contrast, our approach defines the reset condition based on a multi-output function that incorporates all state variables.

3) *Reset Action*: In this paper, we model the reset action, $\mathcal{R}_a(\cdot)$, using a simple element-wise scaling of the state variables v_k , $k = 1, \dots, n$ with a learnable scaling parameter $r_{scale} \in \mathbb{C}^{1 \times 1}$. Hence, 6d becomes

$$v_k[t+1] = \begin{cases} r_{scale} v_k[t+1] & \text{if Reset Condition is True} \\ v_k[t+1] & \text{otherwise} \end{cases} \quad (9)$$

We now compare the reset mechanism in (9) with typical reset mechanisms in neuromorphic computing. In various biologically inspired neuron models, e.g. LIF, the reset condition typically affects only the membrane potential, which is one of the state variables. This process is commonly referred to as the *postsynaptic potential (PSP)*. In contrast to these models, here we propose a generalized change in all state variables when reset occurs. In some models, e.g. LIF, the reset mechanism involves subtracting the threshold value θ from the membrane potential to equate the energy lost from the output spike [7, Eqn.7] [34, Eqn.3]. In contrast, here we consider a generalized reset action that is not directly connected to the threshold for the spiking function. Some models incorporate a *refractory period* [35]–[37], imposing constraints on the behavior of membrane potential after a spike for a certain period. Instead, in (9) an immediate reset action is adopted and the neuron continues to follow the original neuron model.

Remark 3. In the existing popular SNN models, such as LIF and adLIF, the reset action is coupled to the spiking function such that whenever there is a spike out, it gets subtracted from one of the state variables associated with the membrane potential. In our work, we go a step further by introducing trainable reset parameter associated with how much all of the state variables would be scaled.

4) *Neuron Model to Network*: Following the spiking neural networks framework, connections between stateful hidden layers with multiple-output neurons is constructed by connecting output channels across neurons using a dense weight matrix $\mathbf{W} \in \mathbb{R}^{n_{out} \times h}$. This weight matrix parametrizes the connections between spiking outputs from the previous pre-synaptic layer, i.e., $\mathbf{s}_{out}[t]$'s, to the inputs of the next post-synaptic layer, i.e., $\mathbf{i}[t]$'s. See Figure 3 for an illustration. We note that Figure 3 provides a high-level illustration of a network, including the input and output channels of the network. In our experiments, possibly different number of hidden layers are used depending on the dataset. The network structures are discussed in Section V-2.

C. Computational Cost

We now discuss the additional computational cost due to the proposed reset mechanism, focusing on the learnable parameters in Section IV-C1 and number of multiply-accumulate (MAC) operations in Section IV-C2.

1) *Number of learnable parameters*: We now discuss the number of learnable parameters in our proposed neuron model in Section IV-B. Although $\mathbf{A}$ in (6a) can be parametrized as a possibly dense trainable matrix, motivated by the success of models with diagonal state transition matrices [10], [24], [38] and our own numerical results in Section VI, here we focus

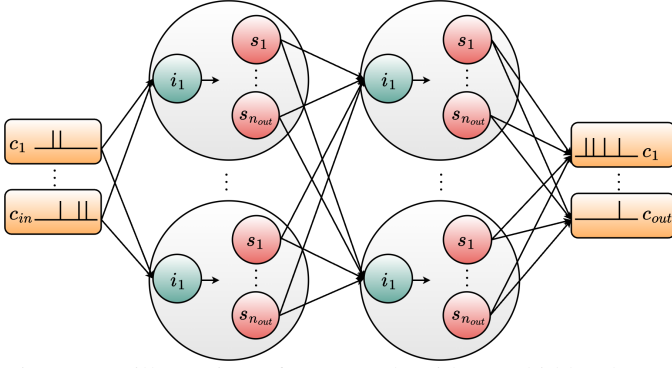


Fig. 3: An illustration of a network with two hidden layers and multiple-output spiking neurons.

on the case $\mathbf{A}$ is diagonal. Therefore, we have n complex-valued parameters for diagonal $\mathbf{A}$. For $\mathbf{B}$ in (6a) we have n real-valued parameters. For $\mathbf{C}$ and c_{bias} in (6b) we have $n \times n_{out}$ complex-valued parameters, and n_{out} complex-valued parameters, respectively. There are no trainable parameters in 6c. For 6d, we have 1 real-valued parameter (r_{bias}) from reset condition (8) and 1 complex-valued parameter (r_{scale}) from reset action (9). Therefore, accounting for the fact that one complex-valued parameter corresponds to two real-valued parameters, our proposed neuron model in Section IV-B has an equivalent of $p = (2n_{out} + 3)(n + 1)$ trainable real-valued parameters. Hence, there are $p \times h$ trainable parameters per hidden layer with h neurons.

The above analysis demonstrates that the reset mechanism adds a small overhead in terms of trainable parameters, introducing only three additional trainable parameters per neuron. In the context of typical values of n and n_{out} in SSM-based deep models and also in our models, see Section VIII-A2, this overhead is not significant.

2) *Number of operations:* We now focus on the additional operations introduced by the reset mechanism per time step.

Evaluating the reset condition (8) requires a total of $m_{rc} = 4n_{out} + 1$ MACs. Specifically, focusing on multiplications/additions, we have $2n_{out}$ real multiplications (i.e. multiplication of real-valued parameters), $2n_{out} - 1$ real additions for calculation of $\|\mathbf{y}[t]\|_2$, 1 multiplication when scaling with $\frac{1}{n_{out}}$, 1 addition with r_{bias} . When triggered, the reset action of (9) scales the state variable, requiring n complex multiplications, each corresponding to 4 real multiplications and 2 real additions, hence $m_{ra} = 6n$ MACs. Hence, assuming a reset occurs, the reset mechanism introduces $m_r = m_{rc} + m_{ra} = 4n_{out} + 6n + 1$ MACs per neuron, and $m_r \times h$ MACs per hidden layer with h neurons.

The computational cost of the reset mechanism is typically negligible compared to the computational cost related to the synaptic weights for a fully connected network architecture. In particular, multiplication with the weight matrix for the synaptic connections from one hidden layer to another hidden layer requires MACs of the order of $h \times n_{out} \times h$. As illustrated in Section VIII-A2, h is typically significantly larger than both n_{out} and n in our models. Hence, focusing on the scaling with h , the hidden layer connections require $\mathcal{O}(h^2)$ operations per time step due to weight matrix operations, whereas the

reset mechanism adds only $\mathcal{O}(h)$ operations. Therefore, the contribution of the reset mechanism to the total computational load is relatively insignificant.

V. EXPERIMENTAL SETUP

In this section, we present the set-up for numerical results. This section provides an overview of the main experimental design choices. Whenever applicable, further details are provided in the Appendix in Section VIII-A.

1) *Neuron Models:* We investigate four neuron model structures based on two key factors: the presence of a reset mechanism and stability of neuron state dynamics. For the reset mechanism, we have the following two cases:

- The neuron resets, i.e., it follows (6a)-(6d)
- The neuron does not reset, i.e., it follows (6a)-(6c).

For the linear part of the neuron state dynamics, we have the following two cases:

- *Stable:* $\mathbf{A}$ is initialized so that the initial neuron dynamics is stable. During training, the eigenvalues are clipped to ensure stability is maintained.
- *Unstable:* $\mathbf{A}$ is initialized so that the initial neuron dynamics is possibly unstable. During training, eigenvalues are not clipped, hence unstable dynamics is allowed.

According to the above, we consider four different neuron models in our experiments: *Stable+Reset*, *Stable+NoReset*, *Unstable+Reset*, and *Unstable+NoReset*. Note that we refer to the linear part of dynamics of the neuron model, i.e. (6a), while using the label *stable* or *unstable*. Note that the stability properties of the overall neuron dynamics may change due to the reset mechanism.

In all experiments, diagonal state transition matrices with learnable eigenvalues are used. The state transition matrices is then $\mathbf{A} = \mathbf{\Lambda}$ where $\mathbf{\Lambda}$ is a diagonal matrix of learnable eigenvalues. Further details for the neuron model, initializations and clipping operations are provided in Section VIII-A1.

2) *Network Architecture:* The number of input channels of a network is determined by the number of data sequences in the dataset. The number of output channels is determined by the number of classes in the classification task. The input and output layers, i.e. the layers consisting of the input and output channels, are state-less. In between the input and output layers, stateful hidden layers whose number depends on the dataset are included, see Figure 3 for a visualization with two hidden layers. The network architecture parameters for different datasets are provided in Section VIII-A. We have trainable synaptic weights between the input layer and the first hidden layer, between the hidden layers, and between the last hidden layer and the output layer. Our networks include batch normalization between layers [39].

In classification tasks, which are the focus of this paper, decisions are made based on the accumulated output spikes. In particular, each output channel corresponds to a specific class, and the class estimated by the network is given by the class with the highest spike count, i.e., rate-based decoding.

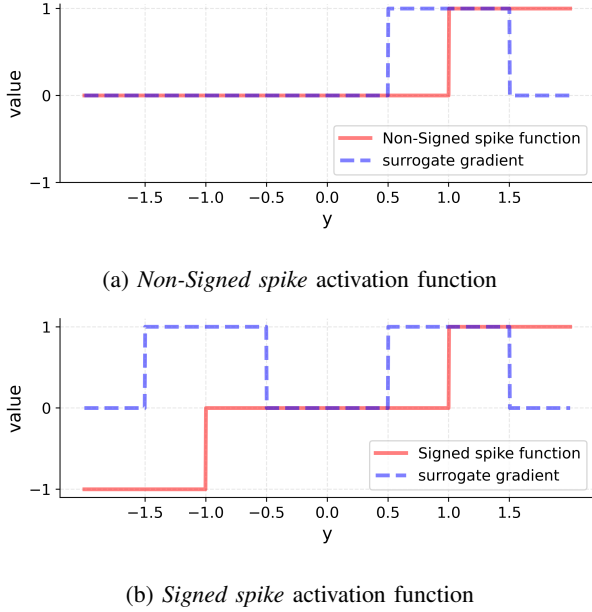


Fig. 4: Visualization of activation function and their surrogate gradient for spiking threshold=1

3) *Activation functions*: The main interest of this paper is binary valued $\{0, 1\}$ information transfer between neurons, i.e. output of the function $f_\theta(\cdot)$ in (6c) is binary-valued. More specifically, we use the *non-signed spikes* (unipolar) illustrated in Figure 4a. We also explore a *signed spikes* (bipolar) spiking function with ternary output values $\{-1, 0, 1\}$. This function visualized in Figure 4b. Compared to the *non-signed spikes*, *signed spikes* increase the level of quantization from 1-bit to 2-bit by including a sign. For both activation functions, surrogate gradient [34] of car-box function at each spiking point is used in the backward pass as approximation to the otherwise non-differentiable activation function, see Figure 4a and Figure 4b. For benchmarking purposes, i.e. in order to quantify the possible performance loss due to low-bit representation with spikes, performance of the continuous-valued *GELU* activation function is also provided in some experiments.

4) *Training*: The models are trained using cross-entropy loss and Batch-propagation through time (BPTT) where surrogate gradient [7], [34] is used for the two heaviside step functions in our approach – the spiking condition of neurons, and the reset condition. Further details are provided in Section VIII-A2.

5) *Datasets*:

a) *Spiking audio processing*: The Multilingual Spoken Word Corpus (MSWC) dataset [12] is a large and growing dataset consisting of spoken words in over 50 languages. In this paper we use a subset of the dataset and its encoding as proposed in the NeuroBench initiative [4]. The subset consists of 100 classes representing different spoken words featuring 20 words per 5 different languages, see Section VIII-A3a for further information. The aim is to classify these 100 words using audio data that is represented using spikes. The current baseline performance for classification accuracy on this subset is 97.1% using continuous-valued ANN [4].

b) *Event-based vision processing*: The DVS-Gesture dataset [13] is one of the most popular benchmarks datasets

for event-based vision processing. It consists of 29 subjects performing 11 hand gestures under 3 different lighting conditions. The dataset is recorded using dynamic vision sensor of 128×128 resolution with two channels encoding the two event polarities. See Section VIII-A3b for further information. The aim is to classify the 11 hand gestures. The current baseline performance for classification accuracy on this dataset is 99.3% using a transformer based architecture [40] and 97.7% with a SSM-based architecture [20].

c) *Sequential pattern recognition*: sMNIST dataset [14] is a popular sequential version of the MNIST dataset [41], providing a popular benchmark for evaluating sequential processing. Each 28×28 image of MNIST corresponds to a handwritten digit, i.e. 0 to 9. In sMNIST, each image is flattened into a time sequence of length $28 \times 28 = 784$. Hence instead of processing all pixels at once as an image, the sample is processed as a time sequence as one pixel at each time step. See Section VIII-A3c for further information. Similar to MNIST, the aim is to classify the 10 digits. The current baseline performance for classification accuracy on this dataset is 99.65% using continuous-valued S5 [11].

VI. RESULTS

In the following sections, we investigate various aspects that influence model behavior and performance. Section VI-A provides an overview of the results. Section VI-B examines the impact of reset mechanisms under stable and unstable dynamics. Section VI-C explores how low-bit resolution communication affects different neuron models. Training dynamics—specifically stability and convergence—are studied in Section VI-D. Section VI-E focuses on time-to-decision behavior, i.e., accuracy progression over time. Section VI-F provides insights into the spiking rates of the models. Section VI-G evaluates the importance and redundancy of multiple output channels. Finally, Section VI-H, analyzes accuracy trade-offs across different combinations of h , n , and n_{out} under architectural constraints for a specific dataset.

All tables report classification accuracy on the test dataset as a percentage, where 100% indicates perfect classification on the test set. These results correspond the test accuracy at the final epoch, and represent the mean accuracy over 5 runs with different random initializations of the network. Standard deviations across these runs are also included.

A. Overview

In Table II we compare the four neuron models of our framework under different activation functions. The table also presents relevant results from the literature, covering both neuroscience-inspired SNNs and models based on the recent deep state-space formulations. We observe that under both spike-based communication and continuous-valued communication, our models achieve accuracy comparable to state-of-the-art methods while maintaining reasonable and even smaller model sizes across all datasets. For instance, our models with the proposed reset mechanism achieve accuracy values close to or matching the best reported results with non-signed spikes as follows: 95.0% (*Stable+Reset*) vs. 95.7% (RadLIF of [29]) on

TABLE II: Classification accuracy (%) for all three datasets for different models with varying the neuron dynamics and amplitude resolution of the activation function. Related literature is grouped into neuroscience-inspired and deep-state-space-inspired, separated with dashed-line. The highest accuracy in our models is in bold; the best from the related work is both bolded and boxed. All accuracy values are rounded to 1 decimal points. Number of parameters for the models in the literature is reported when the original work provides this information. Table VI provides the model hyperparameters.

(a) MSWC dataset.

Model	Dynamics	Reset	<i>Non-signed spikes</i>	<i>Signed spikes</i>	<i>GELU</i>	Num. params.
Ours	Stable	Yes – Proposed	95.0 ± 0.1 %	95.1 ± 0.1 %	95.6 ± 0.1 %	0.4M
Ours	Stable	No	88.9 ± 0.4 %	92.1 ± 0.2 %	95.7 ± 0.0 %	0.4M
Ours	Unstable	Yes – Proposed	91.5 ± 0.4 %	93.9 ± 0.5 %	94.2 ± 0.3 %	0.4M
Ours	Unstable	No	40.2 ± 1.1 %	50.7 ± 1.1 %	90.6 ± 0.4 %	0.4M
RadLIF [4]	Stable	Yes – LIF	93.5 %	-	-	3.4M ^(a)
RadLIF [29]	Stable	Yes – LIF	95.7 %	-	-	3.3M
M5 ANN [4]	N/A	No	-	-	97.1 %	0.5M ^(b) , 1.5M ^(a)

^(a) [4, Table 2], calculated as num. params $\approx$ (Footprint in bytes)/4.

^(b) [42, Table 1].

(b) DVS-Gesture dataset.

Model	Dynamics	Reset	<i>Non-signed spikes</i>	<i>Signed spikes</i>	<i>GELU</i>	Num. params.
Ours	Stable	Yes – Proposed	93.0 ± 1.8	94.0 ± 1.3	97.1 ± 0.3	5.2M
Ours	Stable	No	90.1 ± 3.0	95.4 ± 0.8	92.4 ± 3.8	5.2M
Ours	Unstable	Yes – Proposed	69.0 ± 7.0	84.0 ± 2.2	95.7 ± 0.4	5.2M
Ours	Unstable	No	75.4 ± 14.0	83.5 ± 2.7	95.4 ± 1.3	5.2M
DECOLLE [43]	Stable	Yes-LIF	95.5 %	-	-	-
tdBN [44]	Stable	Yes-LIF	96.9 %	-	-	-
PLIF [8]	Stable	Yes-LIF	97.6 %	-	-	-
EventSSM [20]	Stable	No	-	-	97.7 %	5M

(c) sMNIST dataset.

Model	Dynamics	Reset	<i>Non-signed spikes</i>	<i>Signed spikes</i>	<i>GELU</i>	Num. params.
Ours	Stable	Yes – Proposed	95.8 ± 0.5 %	96.2 ± 0.5 %	95.9 ± 0.6 %	113k
Ours	Stable	No	98.8 ± 0.1 %	99.1 ± 0.1 %	99.3 ± 0.1 %	113k
Ours	Unstable	Yes – Proposed	96.3 ± 0.4 %	97.3 ± 0.4 %	97.2 ± 0.3 %	113k
Ours	Unstable	No	82.9 ± 0.4 %	84.8 ± 0.2 %	95.2 ± 0.2 %	113k
LIF [45]	Stable	Yes – LIF	89.5 % ^(d)	-	-	-
RLIF [45]	Stable	Yes – LIF	96.9 % ^(d)	-	-	-
RadLIF [46]	Stable	Yes – LIF	96.1 % ^(c)	-	-	-
S4D [24]	Stable	No	-	-	99.5 % ^(a)	67k
S5 [11]	Stable	No	-	-	99.7 %	-
Q-S5 [23]	Stable	No	-	-	99.6 % ^(b)	-
SpikingSSM [27]	Stable	Yes – LIF output	99.6 %	-	-	-
BinarySSM [26]	Stable	Yes – LIF output	99.4 %	-	-	118k

^(a) RELU instead of GELU used.

^(b) 8-bit quantized GELU (qGELU) used.

^(c) Reported best accuracy run.

^(d) Read from [45, Fig. S5].

MSWC, 93.0% (*Stable+Reset*) vs. 97.6% (PLIF [8]) on DVS-Gesture, and 96.3 % (*Unstable+Reset*) vs. 99.6% (SpikingSSM [27]) on sMNIST.

These results indicate that our proposed model generalizes well across modalities, including audio, vision and sequential pattern recognition tasks. These results also illustrate that our proposed model can handle relatively long-sequences as input. Notably, DVS-Gesture (with an average of ≈ 340 time steps, see Section VIII-A3b) and sMNIST (with 784 time steps) has samples with relatively long-sequences in comparison to many other standard datasets in the neuromorphic computing literature, such as the Spiking Heidelberg Digits (SHD) dataset and Spiking Speech Command (SSC), which are often used with 100 time steps [7], [28], [45].

All subsequent results use our models from Table II unless otherwise stated explicitly.

B. Impact of reset under stable and unstable dynamics

We now focus on Table II and compare the four neuron models (*Stable* or *Unstable*, *Reset* or *NoReset*) under our proposed framework.

For *Stable* dynamics, how the reset affects the performance depends on the dataset and the activation function. For example, under *non-signed spikes*, for the MSWC dataset and DVS-Gesture dataset it increases the accuracy from 88.9% to 95.0% and 90.1% to 93.0%, respectively. On the other hand, for the sMNIST dataset, it drops the performance from 98.8% to 95.8%.

For *Unstable* dynamics, using reset increases the performance for MSWC and sMNIST. The performance improvement is large for the low bit communication cases of *non-signed and signed spikes* whereas it is relatively smaller under the continuous-valued communication case of GELU. For example, an increase from 40.2% to 91.5% for MSWC, and 82.9% to

96.3% for sMNIST is observed under *non-signed spikes*. For DVS-Gesture, the performance changes are within standard deviation limits where the performance has large fluctuations under *non-signed spikes*, suggesting further regularization and reset optimization may be useful.

We note the results in the table for unstable dynamics without reset are obtained with clipping of the state and clipping of gradients. Without clipping, the computation cannot be performed in this scenario due to diverging states and exploding gradients, see Section VIII-A1. The results in the table illustrate that incorporating reset has the potential to stabilize learning and to improve the performance.

C. Low-bit resolution in neuron communications

We now consider Table II from the perspective of the impact of quantization of the neuron output, i.e. the amplitude resolution in the output of activation function, hence the communication between the neurons. We recall that *non-signed spikes* and *signed spikes* correspond to 1 and 2 bit representation in each neuron output whereas in *GELU*, the output is continuous-valued. Here, the continuous-valued *GELU* is included for benchmarking purposes.

Focusing on the mean accuracy values in our models, we observe that a performance improvement is observed when *signed spikes* are used instead of *non-signed spikes* for all cases. Nevertheless, for many scenarios, the difference in the accuracy is smaller than 2%, for example MSWC under *Stable+Reset*, sMNIST under *Stable+Reset*, *Stable+NoReset*, *Unstable+Reset*. Hence, the extra bit provided by the *signed spiking* has not been enough to provide a large meaningful performance improvement in these cases.

For *Stable+Reset*, the relative performance of *non-signed* and *signed* spiking models compared to that of *GELU* models depends on the dataset. For MSWC and sMNIST, the performance is comparable in many cases demonstrating that even with 1-bit quantization (as in the *non-signed* case), competitive results can still be achieved. For DVS-Gesture, the gap between the best performing models under *spiking* activation functions and *GELU* is slightly larger, i.e., 95.4% and 97.1%, respectively.

For *Unstable+NoReset*, *GELU* still performs reasonably well compared to the low-bit spiking neuron models. This suggests that the communication between neurons, i.e., the information transfer between neurons, may be used to compensate the negative effect of unstable dynamics. In particular, the continuous-valued activation function of *GELU* allows more information to flow between neurons, which possibly compensates for instability and enables the model to learn relatively effectively.

D. Training stability and convergence

We now explore the impact of reset mechanism and dynamics on the training behavior of the models. Specifically we look at the convergence and smoothness of accuracy curves in Figure 5. In Figure 5, test classification accuracy is plotted against the number of training epochs, and the standard deviations across runs are indicated with shaded areas.

First, we focus on the *Stable* dynamics, and compare the *Stable+Reset* and *Stable+NoReset* models. For the MSWC

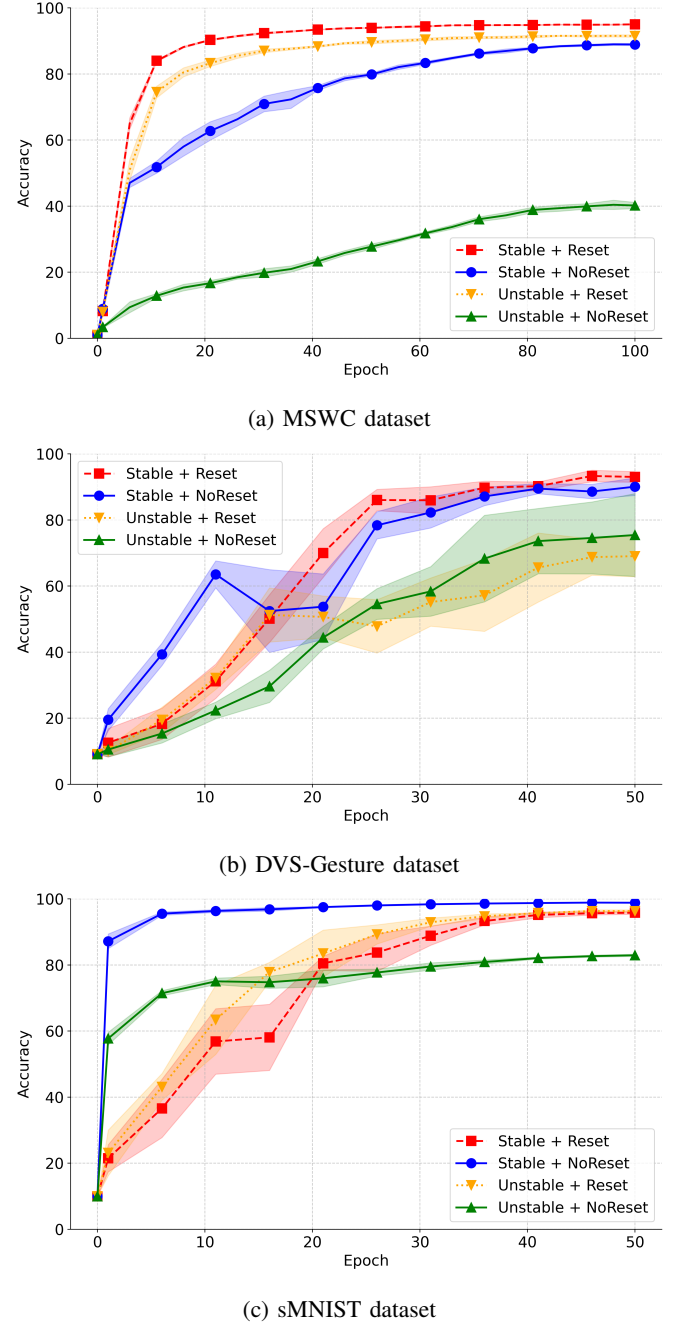


Fig. 5: Test classification accuracy over training epochs for *non-signed spikes*.

dataset, the model with reset converges faster. For the DVS-Gesture dataset, the model with no reset achieves higher accuracy values earlier during training but then the accuracy values drop to lower values while training continues. Eventually, the model with reset surpasses the model without reset. For the sMNIST dataset, model without reset converges faster. These results suggest that the impact of the reset mechanism on the convergence during training depends on the characteristics of data.

We now focus on the *Unstable* dynamics, and compare the *Unstable+Reset* and *Unstable+NoReset* models. For both MSWC and sMNIST, we observe a substantial gap in favor of the model with reset in terms of the accuracy values the

curves converge to. For DVS-Gesture, the models with reset and without reset perform within each other's standard deviation range. Similar to *Stable* case, the rate of convergence depends on the dataset with no clear trend disfavoring the models with reset.

Across all datasets and for both *Stable* and *Unstable* models, the accuracy curves in general monotonically increases under reset. The only clearly non-monotonic curve is that of *Stable+NoReset* on DVS-Gesture. We now consider the initialization dependence, i.e. the standard deviation ranges across different runs during training. For the MSWC dataset, there is no significant difference between models with reset and without reset. For the DVS-Gesture dataset all models have the same level of variability for a given type of dynamics, i.e. *Stable* or *Unstable*. For the sMNIST dataset, models with reset have a higher variability. Hence, in terms of variation across runs, the presence of reset does not lead to a consistent increase in variability compared to corresponding models without reset.

In conclusion, although our separate reset mechanism introduces an additional non-linear discontinuous data processing block into the spiking neuron, our experiments do not provide any strong evidence that the inclusion of our reset mechanism significantly deteriorates the convergence speed or training stability. The impact appears to be data-dependent, suggesting that the proposed reset mechanism does not inherently make training more difficult or unstable.

E. Accuracy over time

In this section, we explore the impact of reset mechanism and dynamics on the accuracy over time. Specifically we look at how the test classification accuracy changes as the models see a larger part of the input sequences. We focus on Figure 6 where the test accuracy is plotted against time steps.

First, we focus on the *Stable* dynamics, and compare the *Stable+Reset* and *Stable+NoReset* models. We observe that for both the MSWC and sMNIST dataset, models with reset achieves reasonable accuracy values faster. Specifically, if we consider the time step at which the models reach 80% accuracy for *Reset* vs. *NoReset*, we have $t = 40$ vs. $t = 45$ for MSWC, and $t = 500$ vs. $t = 640$ for sMNIST. This demonstrates a clear benefit of including the reset mechanism for these datasets if quick decisions at these accuracy levels are desired. For the DVS-Gesture dataset, the performance of *Reset* and *NoReset* configurations is comparable, showing no significant difference.

Second, we focus on the *Unstable* dynamics, and compare the *Unstable+Reset* and *Unstable+NoReset* models. For MSWC and sMNIST, we observe a substantial accuracy gap in favor of the reset configuration at the end of time sequence. Even setting aside this final accuracy gap, we observe that models with reset consistently begins improving earlier compared to the models without reset. For DVS-Gesture, the difference in the behavior of the models with and without reset are within the standard deviation ranges of each other.

In conclusion, these experiments illustrate that including the reset can either significantly improve the time to reach a reasonable level of accuracy, or does not affect it negatively.

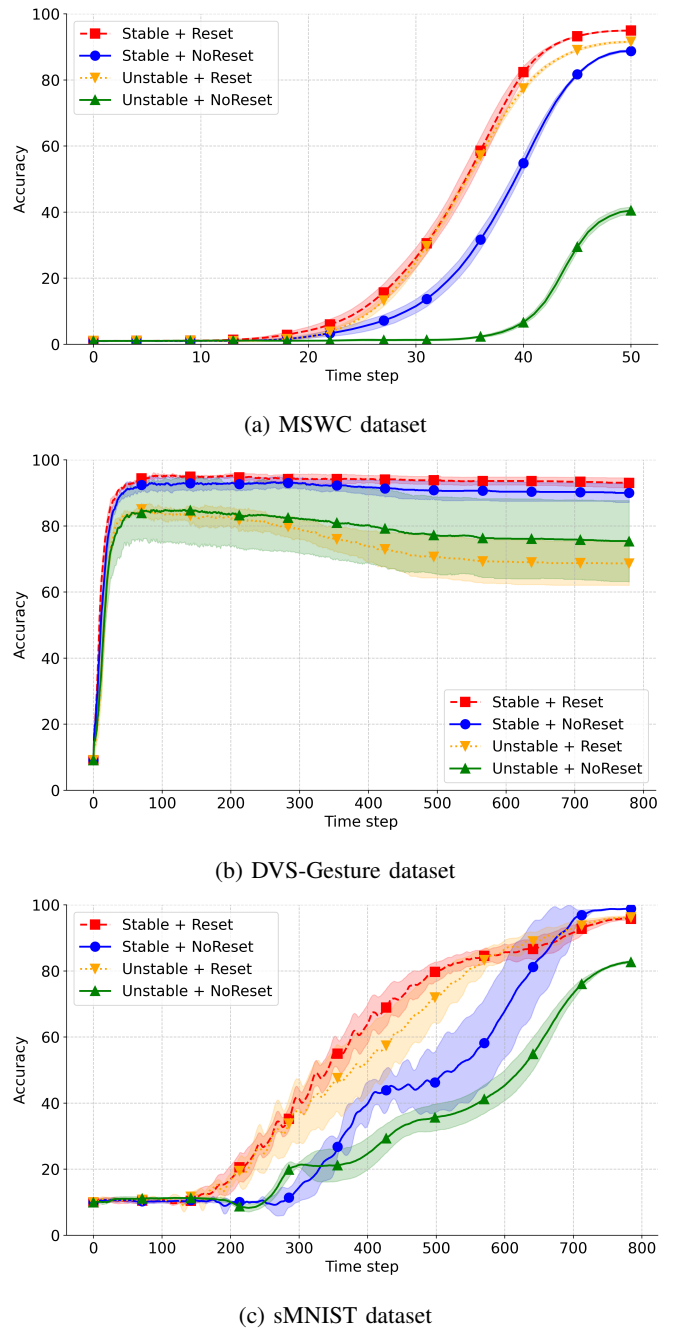


Fig. 6: Test classification accuracy over time-steps of the input sequences for *non-signed spikes*.

F. Spike rate

We now investigate the spike rate for the models with spiking activation function. At each time step, each spiking neuron either provides no output, or emits a spike, either always with a value of 1, as in *non-signed spikes*, or with values of 1 or -1 , as in *signed spikes*. In this experiment, we focus on spike activity regardless of its value and calculate the spike rate over time for all neurons in a hidden layer and calculate its mean and standard deviation. The values in Table III represent the mean of these values for our models from Table II.

In particular, we observe two cases where the mean spike rate is significantly lower when reset is used, compared to configurations without reset: for MSWC, *signed spikes* and

TABLE III: Spike rate per hidden layer, rounded to 2 decimal point.

(a) MSWC dataset

Dynamics	Reset	Non-signed spikes		Signed spikes	
		layer 1	layer 2	layer 1	layer 2
Stable	Yes	0.42 ± 0.15	0.40 ± 0.16	0.69 ± 0.15	0.78 ± 0.16
Stable	No	0.46 ± 0.28	0.40 ± 0.31	0.98 ± 0.03	0.98 ± 0.04
Unstable	Yes	0.49 ± 0.15	0.37 ± 0.16	0.70 ± 0.15	0.75 ± 0.16
Unstable	No	0.50 ± 0.21	0.49 ± 0.20	0.98 ± 0.03	0.99 ± 0.01

(b) DVS-Gesture dataset

Dynamics	Reset	Non-signed spikes			Signed spikes		
		layer 1	layer 2	layer 3	layer 1	layer 2	layer 3
Stable	Yes	0.50 ± 0.27	0.50 ± 0.26	0.50 ± 0.28	0.80 ± 0.20	0.92 ± 0.14	0.98 ± 0.09
Stable	No	0.50 ± 0.28	0.50 ± 0.26	0.50 ± 0.28	0.74 ± 0.23	0.85 ± 0.19	0.95 ± 0.12
Unstable	Yes	0.50 ± 0.28	0.50 ± 0.27	0.50 ± 0.28	0.66 ± 0.25	0.81 ± 0.22	0.92 ± 0.15
Unstable	No	0.50 ± 0.27	0.50 ± 0.28	0.50 ± 0.30	0.65 ± 0.25	0.80 ± 0.22	0.90 ± 0.17

(c) sMNIST dataset

Dynamics	Reset	Non-signed spikes		Signed spikes	
		layer 1	layer 2	layer 1	layer 2
Stable	Yes	0.46 ± 0.30	0.49 ± 0.22	0.71 ± 0.30	0.77 ± 0.23
Stable	No	0.50 ± 0.12	0.50 ± 0.09	0.97 ± 0.04	0.98 ± 0.03
Unstable	Yes	0.48 ± 0.24	0.49 ± 0.16	0.80 ± 0.22	0.84 ± 0.17
Unstable	No	0.50 ± 0.09	0.50 ± 0.07	1.00 ± 0.00	1.00 ± 0.00

Stable configuration, where including the reset reduced the spike rate from 0.98 to 0.69 in layer 1, and 0.99 to 0.75 in layer 2. In all other cases examined, no significant difference is observed i.e., the mean values are within one standard deviation range of each other. We note that no explicit regularization on spike rate was imposed during training. Nevertheless, the results suggest that, across the studied datasets, the proposed reset mechanism alone either reduces the spike rate or leaves it unchanged.

G. Multiple output channels in spiking neurons

We now investigate whether the output channels of neurons are equally informative or exhibit redundancy.

To investigate this aspect, we perform a series of experiments where a number of output channels of each neuron are dropped, i.e. the output of the neuron for that output channel is set to 0 at all time steps. The channels that are dropped are selected based on their indices using either the “First” or “Last” indices. For instance, if 2 channels are dropped out of n_{out} channels, then the channels with indices 1 and 2 are dropped under “First”, while the channels with indices $n_{out} - 1$ and n_{out} are dropped under “Last”. We use the pretrained models of Table II for *Stable+Reset* with *non-signed* spikes for each dataset. We present the resulting accuracy values of this experiment in Table IV.

Across all datasets, we observe that removing a single output channel per neuron leads to a noticeable drop in accuracy, which becomes more pronounced when two channels are removed. This suggests that each output channel captures complementary features that cannot be fully recovered by the others. We now compare the results across different rows, i.e., when varying which specific output channel(s) are removed based on their index. Considering the standard deviations, we observe that the performance remains within the same accuracy interval in each case. This suggests that all output channels are similarly

TABLE IV: Classification accuracy (%) under drop-out of output channels of neurons for the models configuration *Stable+Reset* under *non-signed* spikes

(a) MSWC dataset. Recall $n_{out} = 4$, and the baseline from Table IIa, which uses all output channels of neurons is $95.0 \pm 0.1\%$.

Which	How many	drop 1 channel	drop 2 channels
First		$48.0 \pm 9.5 \%$	$4.3 \pm 1.5 \%$
Last		$46.7 \pm 4.2 \%$	$3.8 \pm 2.0 \%$

(b) DVS-Gesture dataset. Recall $n_{out} = 24$, and the baseline from Table IIb, which uses all output channels of neurons is $93.0 \pm 1.8\%$.

Which	How many	drop 1 channel	drop 2 channels
First		$79.0 \pm 7.4 \%$	$65.9 \pm 5.3 \%$
Last		$70.9 \pm 7.2 \%$	$63.5 \pm 14.1 \%$

(c) sMNIST dataset. Recall $n_{out} = 8$, and the baseline from Table IIc, which uses all output channels of neurons is $95.8 \pm 0.5\%$.

Which	How many	drop 1 channel	drop 2 channels
First		$40.7 \pm 11.8 \%$	$19.6 \pm 9.8 \%$
Last		$40.4 \pm 17.5 \%$	$26.4 \pm 13.8 \%$

important and contribute comparably to the network’s overall performance.

H. Impact of network architecture

We now investigate the effect of network architecture. For illustration, we focus on the MSWC dataset and provide the results in Table V for *non-signed* spikes.

The architectures in Table V are designed as follows: For fair comparison in terms of layer width and network depth, we consider the SNN architectures in [4], [29] for MSWC dataset as a starting point. In particular, both works use SNN models of 2 hidden layers with $h = 1024$ adLIF neurons per hidden layer, where each neuron has two state variables, i.e., $n = 2$, and has a single output, i.e., $n_{out} = 1$. Hence, in all experiments in Table V, we use 2 hidden layers and explore the trade-offs between different layer architectures in terms number of neurons h , the number of neuron state dimension n ,

TABLE V: Classification accuracy (%) on the MSWC dataset for different models by varying the neuron dynamics and dimensions. The best accuracy over all models is in bold.

(a) Models have $h = 1024$ neurons per hidden layer, each with $n = 2$ state variables. Here, $n_{out} = 1$ satisfies $h \times n_{out} = 1024$.

Dynamics	Reset	$n_{out} = 1$	$n_{out} = 4$	$n_{out} = 8$	$n_{out} = 16$
Stable	Yes	95.2 ± 0.2 %	95.7 ± 0.1 %	95.8 ± 0.1 %	95.7 ± 0.1 %
Stable	No	93.8 ± 0.1 %	95.3 ± 0.1 %	95.6 ± 0.0 %	95.8 ± 0.1 %
Unstable	Yes	91.6 ± 0.2 %	93.5 ± 0.1 %	93.3 ± 0.4 %	93.9 ± 0.2 %
Unstable	No	79.1 ± 0.6 %	84.4 ± 0.2 %	86.8 ± 0.3 %	88.4 ± 0.4 %

(b) Models have $h = 256$ neurons per hidden layer, each with $n = 8$ state variables. Here, $n_{out} = 4$ satisfies $h \times n_{out} = 1024$.

Dynamics	Reset	$n_{out} = 1$	$n_{out} = 4$	$n_{out} = 8$	$n_{out} = 16$
Stable	Yes	89.3 ± 0.7 %	94.9 ± 0.1 %	95.2 ± 0.1 %	95.4 ± 0.1 %
Stable	No	78.0 ± 0.6 %	89.0 ± 0.2 %	91.0 ± 0.1 %	92.5 ± 0.3 %
Unstable	Yes	81.0 ± 1.1 %	91.6 ± 0.3 %	94.4 ± 0.3 %	94.6 ± 0.4 %
Unstable	No	20.7 ± 1.1 %	40.0 ± 0.7 %	49.9 ± 0.9 %	59.2 ± 0.5 %

(c) Models have $h = 128$ neurons per hidden layer, each with $n = 16$ state variables. Here, $n_{out} = 8$ satisfies $h \times n_{out} = 1024$.

Dynamics	Reset	$n_{out} = 1$	$n_{out} = 4$	$n_{out} = 8$	$n_{out} = 16$
Stable	Yes	83.3 ± 1.0 %	93.5 ± 0.3 %	94.5 ± 0.2 %	95.0 ± 0.1 %
Stable	No	63.4 ± 2.3 %	85.0 ± 0.2 %	88.3 ± 0.3 %	90.4 ± 0.2 %
Unstable	Yes	69.7 ± 1.8 %	86.8 ± 1.0 %	92.4 ± 1.2 %	94.4 ± 0.3 %
Unstable	No	8.3 ± 0.4 %	19.3 ± 0.8 %	26.6 ± 0.6 %	35.1 ± 0.7 %

(d) Models have $h = 64$ neurons per hidden layer, each with $n = 32$ state variables. Here, $n_{out} = 16$ satisfies $h \times n_{out} = 1024$.

Dynamics	Reset	$n_{out} = 1$	$n_{out} = 4$	$n_{out} = 8$	$n_{out} = 16$
Stable	Yes	70.1 ± 2.9 %	90.3 ± 0.3 %	92.8 ± 0.3 %	94.0 ± 0.2 %
Stable	No	42.7 ± 1.4 %	77.5 ± 1.0 %	84.2 ± 0.6 %	87.4 ± 0.3 %
Unstable	Yes	53.3 ± 2.4 %	77.8 ± 1.8 %	85.4 ± 1.0 %	91.8 ± 1.3 %
Unstable	No	3.7 ± 0.4 %	8.2 ± 0.3 %	13.8 ± 0.9 %	19.3 ± 0.7 %

and number of output channels per neuron n_{out} . We consider two design rules as reference points:

- Fixed latent space dimension: number of total state variables in a hidden layer is fixed as $h \times n = 2048$
- Fixed number of outgoing synaptic connections: number of outgoing synaptic connections from a hidden layer is fixed as $h \times n_{out} = 1024$

In Table V, every model has a constant number of state variables in a layer, $h \times n = 2048$. Some of the models further satisfy $n_{out} \times h = 1024$, as indicated in the captions.

Table V illustrates that there are multiple network architectures that achieve comparable performance under the proposed reset mechanism. For instance, all the network architectures with $h = 1024$, $n = 2$ under *Stable* with the proposed reset obtain above 95% average accuracy. Table V also illustrates that the proposed reset mechanism may provide performance improvements over a wide range of network architectures. For *Stable* case, inclusion of reset results in either a change in the performance within the standard deviation range, or an improvement of up to $\approx 38\%$ where the magnitude of improvement is typically larger with lower n_{out} and lower h . For the *Unstable* case, it is crucial to use reset to obtain performance levels comparable with the *Stable* case. Comparing *Unstable+Reset* with *Unstable+NoReset*, we observe that inclusion of reset introduces a performance improvement in the range $\approx [5\%, 60\%]$, where again, the magnitude of improvement is typically larger with lower n_{out} and lower h .

VII. DISCUSSIONS AND CONCLUSIONS

This work proposes a multiple-output, stateful spiking neuron model with a general linear state transition dynamics and non-linear reset-based feedback mechanism. Inspired by neuromor-

phic computing principles, the proposed model incorporates spike-based communication, threshold mechanisms, and state discharging based on its output.

Our formulation generalizes the reset mechanisms in SNN literature by providing a clear formulation that distinguishes the spiking function, the reset condition, and the reset action. Our results illustrate that competitive performance can be obtained using the proposed neuron model.

An important aspect of the proposed reset mechanism is its role in the stability of the neuron, and hence stability of learning in the neural network. In contrast to the existing works that enforce stable linear SSM transition, our proposed neuron model allows instability in the linear part of the state transition. In particular, our results illustrate that for models with unstable linear dynamics and low-bit information processing, i.e., activation functions whose outputs are quantized with a low number of bits, reset mechanism may make learning possible even when the trainable network parameters diverge and the network fails to learn without the reset mechanism. Our results showed that across different datasets and for a wide range of network architectures reset contributed to achieving results comparable with the benchmark of continuous-valued information processing. Hence, our framework provides a promising starting point for investigation of a richer family of SSM models in neural networks.

We now discuss the implementation of the proposed neuron models on neuromorphic hardware. LIF type of reset mechanism has been implemented on multiple neuromorphic chips such as Loihi [3], TrueNorth [47]. By representing complex-valued matrices with real-valued equivalents, feasibility of deploying models with complex-valued diagonal state-space transition matrices on Loihi has been recently demonstrated

[24]. However, whether our proposed model, which combines a reset mechanism that goes beyond the reset mechanism in LIF, and complex-valued state-space models, may be efficiently deployed on neuromorphic hardware remains an open question. Investigation of feasibility of such an implementation, and the resulting accuracy, computational complexity and energy trade-offs is a key direction for future work.

A fundamental question is to what extent the reset mechanism can prevent divergence in learning under neurons with an unstable dynamics and low-bit activation functions. Quantification of these limits is considered a promising future research direction. Another future research direction is exploring fully learnable reset conditions. This article investigated a reset mechanism where the reset condition is given by a norm condition on the state variable with a learnable effective threshold parameter and the reset action is performed using linear scaling with a learnable parameter. More general forms of reset conditions and actions, where the parameters of broader family of functions are learned from data, is considered a promising research direction.

VIII. APPENDIX

A. Further Details of the Set-up for Numerical Results

1) *SSM-Neuron Parametrization*: The diagonal matrix Λ is initialized using the S4D-Lin initialization with bilinear discretization [10], where n different eigenvalues are initialized without explicit repetition of complex conjugates.

For stable $\mathbf{A}$, this initialization of Λ is used with an additional clipping of the modulus of eigenvalues so that they satisfy ≤ 1 while training in order to ensure stable discrete system/neuron. On the other hand, for unstable $\mathbf{A}$, after the initialization of Λ , we perform additional step by multiplying every second eigenvalues by 1.5. This results in unstable system/neuron where half of the eigenvalues of $\mathbf{A}$ have modulus > 1 . Note due to the delta range in the initialization of Λ [10], modulus of the eigenvalues ≈ 0.9 , and hence multiplying the eigenvalues with 1.5 results in modulus of eigenvalues > 1 , hence unstable system/neuron. In unstable dynamics, we do not perform clipping of eigenvalues while training, allowing the eigenvalues of $\mathbf{A}$ to settle for possibly unstable values.

For unstable $\mathbf{A}$, $\|v\|$ possibly increases in unbounded fashion. If no action is taken, this typically leads to very large numbers in the state, and prevents learning completely. Hence, we clip as $|v_k| < 1000$, which prevents exploding of state values while training.

Following the S4D-Lin initialization with bilinear discretization [10], $\mathbf{C}$ is initialized from the standard normal distribution and $\mathbf{B}$ is fixed as a real-valued matrix of ones. We initialize $\mathbf{c}_{bias}$ with zeros. Hence, the trainable parameters in the linear part of the SSM are Λ , $\mathbf{C}$ and $\mathbf{c}_{bias}$.

The output of the state evolution $\mathbf{y}$ is complex-valued. We use the transformation $\mathbf{y} \rightarrow \Re(\mathbf{y}) + \Im(\mathbf{y})$ before applying the activation function $f(\cdot)$.

2) *Training*: Models are trained on NVIDIA Tesla T4 GPUs with 16GB RAM. Due to computational constraints, limited hyperparameters optimization (HPO) is performed and hyperparameters based on a validation subset are chosen,

TABLE VI: Hyperparameters used for each dataset.

Parameter	MSWC	DVS-Gesture	sMNIST
c_{in}	20	32768	1
c_{out}	100	11	10
Nb. Hidden Layers	2	3	2
h	256 (*)	128	96
n	8 (*)	8	8
n_{out}	4 (*)	24	8
LR	See Table VII	10^{-4}	10^{-3}
LR-SSM	See Table VII	10^{-2}	10^{-4}
LR- r_{scale}	=LR	10^{-4}	10^{-5}
LR- r_{bias}	=LR	10^{-6}	10^{-5}
WD	See Table VII	0	10^{-2}
WD-SSM	10^{-3}	10^{-2}	10^{-3}
WD- r_{scale}	=WD	10^{-2}	0
WD- r_{bias}	=WD	10^{-5}	0
r_{scale} initialization	0.1	0.8	0.5
r_{bias} initialization	0	0	0
Nb. Epochs	100	50	50
Batch Size	256	16	128
Dropout	0.4	0	0.3

(*) The given value is used for all MSWC results in Section VI except in Table V, where it is explicitly stated otherwise.

TABLE VII: Additional hyperparameters used for MSWC dataset.

Parameter	Stable-Reset	Stable-No-Reset	UnStable-Reset	UnStable-NoReset
LR	10^{-3}	10^{-2}	10^{-3}	10^{-3}
LR-SSM	10^{-3}	10^{-2}	10^{-3}	10^{-3}
WD	10^{-3}	10^{-4}	10^{-4}	10^{-4}

see Table VI for the resulting values. For sMNIST and DVS-Gesture datasets, HPO was performed for the case of *Stable+NoReset* and GELU activation function, and the chosen hyperparameters are used for all other model cases and activation functions. For the reset-dependent hyperparameters (i.e. initial values of r_{scale} and r_{bias} and related learning rates), an additional HPO is performed on *Stable+Reset* and GELU activation function and the chosen hyperparameters are used for all other model cases with reset and activation functions. For MSWC dataset, HPO was performed for the cases of (i) *Stable+Reset*, (ii) *Stable+NoReset* and (iii) *Unstable+Reset*. For each case, the hyperparameters evaluated are the base learning rates, weight decay regularization parameters, n and n_{out} . In addition, for the *Reset* case the initial value of r_{scale} and r_{bias} are considered. We do not perform explicit search for LR and WD parameters related to r_{scale} and r_{bias} , but instead use the LR-others and WD-others, respectively. The choices from (iii) are also used for *Unstable+NoReset*. For all HPO studies of MSWC, we use *non-signed spikes* activation function. The same hyper-parameter set is used for all multiple-output dimensions scenarios in Table V and all activation functions in Table IIa.

In all models, ADAMW optimizer and cosine learning rate scheduler are adopted. To prevent vanishing or exploding gradients which might occur while training, we clip the gradients' absolute value to 10^5 .

3) Datasets:

a) *MSWC*: For each class 500 train, 100 validation and 100 test samples are used. Each audio sample is converted to signed ternary sequence i.e. a sequence of values $-1, 0, 1$ using the Speech2Spikes (S2S) [48] preprocessing algorithm with 20 input channels and 201 time steps [4]. The data is further

processed by sum-binning with a non-overlapping moving window along its time dimension with bin-size of 4.

b) *DVS-Gesture*: The dataset has 1342 samples divided into 256 test and 1086 train samples. To improve generalization, we follow EventSSM's data augmentation transformations [20], and use the convention of binning events into time-frames. We follow the standard convention of aggregating raw events into frames by binning them over fixed time windows. Specifically, we use the framework of [49] with a bin size of $\Delta = 20\text{ms}$, which aligns with values commonly used in the literature—for instance, [50] use $\Delta = 25\text{ms}$, and [51] use up to $\Delta = 20\text{ms}$. As a result, the samples of DVS-Gesture have a varying time-sequence length, an average of 340 time steps. For hyperparameter optimization, we used 10% of the training set as a validation set.

c) *sMNIST*: MNIST consists of 60 000 training and 10 000 testing samples. Since sMNIST is a sequential variant of MNIST, the same test and train split is used. For hyperparameter optimization, we used 10% of the training set for validation.

REFERENCES

- [1] S. Sudhakar, V. Sze, and S. Karaman, "Data centers on wheels: Emissions from computing onboard autonomous vehicles," *IEEE Micro*, vol. 43, no. 1, pp. 29–39, 2023.
- [2] B. Rajendran, A. Sebastian, M. Schmuker, N. Srinivasa, and E. Eleftheriou, "Low-power neuromorphic hardware for signal processing applications: A review of architectural and system-level design approaches," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 97–110, 2019.
- [3] M. Davies *et al.*, "Advancing neuromorphic computing with Loihi: A survey of results and outlook," *Proceedings of the IEEE*, vol. 109, no. 5, pp. 911–934, 2021.
- [4] J. Yik *et al.*, "Neurobench: A framework for benchmarking neuromorphic computing algorithms and systems," 2024.
- [5] W. Gerstner and W. M. Kistler, *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press, 2002.
- [6] M. Baronig, R. Ferrand, S. Sabathiel, and R. Legenstein, "Advancing spatio-temporal processing in spiking neural networks through adaptation," 2024. [Online]. Available: <https://arxiv.org/abs/2408.07517>
- [7] A. Bittar and P. N. Garner, "A surrogate gradient spiking baseline for speech command recognition," *Frontiers in Neuroscience*, vol. 16, 2022.
- [8] W. Fang, Z. Yu, Y. Chen, T. Masquelier, T. Huang, and Y. Tian, "Incorporating learnable membrane time constant to enhance learning of spiking neural networks," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2021, pp. 2661–2671.
- [9] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," 2022.
- [10] A. Gu, K. Goel, A. Gupta, and C. Ré, "On the parameterization and initialization of diagonal state space models," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 35 971–35 983, 2022.
- [11] J. T. Smith, A. Warrington, and S. Linderman, "Simplified state space layers for sequence modeling," in *The Eleventh Inter. Conf. on Learning Representations (ICLR)*, 2023.
- [12] M. Mazumder *et al.*, "Multilingual spoken words corpus," in *Conf. on Neural Inf. Process. Systems (NeurIPS)*, 2024.
- [13] A. Amir *et al.*, "A low power, fully event-based gesture recognition system," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, July 2017.
- [14] Q. V. Le, N. Jaitly, and G. E. Hinton, "A simple way to initialize recurrent networks of rectified linear units," *arXiv preprint arXiv:1504.00941*, 2015.
- [15] S. Karilanova, S. Dey, and A. Özçelikkale, "State-space model inspired multiple-input multiple-output spiking neurons," in *2025 Neuro Inspired Computational Elements (NICE)*, 2025.
- [16] A. Lazar and L. Toth, "Perfect Recovery and Sensitivity Analysis of Time Encoded Bandlimited Signals," *IEEE Tran. on Circuits and Systems I*, 2004.
- [17] H. Naaman, S. Mulleti, and Y. C. Eldar, "FRI-TEM: Time encoding sampling of finite-rate-of-innovation signals," *IEEE Transactions on Signal Processing*, vol. 70, pp. 2267–2279, 2022.
- [18] N. T. Thao, D. Rzepka, and M. Miśkiewicz, "Bandlimited signal reconstruction from leaky integrate-and-fire encoding using POCS," *IEEE Transactions on Signal Processing*, vol. 71, pp. 1464–1479, 2023.
- [19] K. Adam, A. Scholefield, and M. Vetterli, "Asynchrony increases efficiency: Time encoding of videos and low-rank signals," *IEEE Transactions on Signal Processing*, vol. 70, pp. 105–116, 2022.
- [20] M. Schöne, N. M. Sushma, J. Zhuge, C. Mayr, A. Subramoney, and D. Kappel, "Scalable event-by-event processing of neuromorphic sensory signals with deep state-space models," *IEEE/ACM International Conference on Neuromorphic Systems (ICONS)*, 2024.
- [21] T. Soydan, N. Zubić, N. Messikommer, S. Mishra, and D. Scaramuzza, "S7: Selective and simplified state space layers for sequence modeling," 2024.
- [22] H.-Y. Chiang, C.-C. Chang, N. Frumkin, K.-C. Wu, and D. Marculescu, "Quamba: A post-training quantization recipe for selective state space models," 2024.
- [23] S. Abreu, J. E. Pedersen, K. M. Heckel, and A. Pierro, "Q-s5: Towards quantized state space models," 2024.
- [24] S. M. Meyer *et al.*, "A diagonal structured state space model on Loihi 2 for efficient streaming sequence processing," 2024.
- [25] M. Bal and A. Sengupta, "Rethinking spiking neural networks as state space models," 2024.
- [26] M.-I. Stan and O. Rhodes, "Learning long sequences in spiking neural networks," *Scientific Reports*, vol. 14, no. 1, p. 21957, 2024.
- [27] S. Shen *et al.*, "Spikingssms: Learning long sequences with sparse and parallel spiking state space models," 2024.
- [28] M. Fabre, L. Dudchenko, and E. Neftci, "Structured state space model dynamics and parametrization for spiking neural networks," 2025. [Online]. Available: <https://arxiv.org/abs/2506.06374>
- [29] S. Karilanova, M. Fabre, E. Neftci, and A. Özçelikkale, "Zero-shot temporal resolution domain adaptation for spiking neural networks," 2024. [Online]. Available: <https://arxiv.org/abs/2411.04760>
- [30] M. Davies *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.
- [31] E. M. Izhikevich, "Simple model of spiking neurons," *IEEE Transactions on neural networks*, vol. 14, no. 6, pp. 1569–1572, 2003.
- [32] A. Hodgkin and A. Huxley, "A quantitative description of membrane current and its application to conduction and excitation in nerve," *Journal of Physiology*, vol. 117, pp. 500–544, 1952.
- [33] Z. Gajic, *Linear Dynamic Systems and Signals*. Prentice Hall - Pearson Education Inc., 2003.
- [34] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," *IEEE Signal Process. Magazine*, vol. 36, pp. 51–63, 2019.
- [35] F. Zenke and S. Ganguli, "Superspike: Supervised learning in multilayer spiking neural networks," *Neural Computation*, vol. 30, no. 6, p. 1514–1541, jun 2018.
- [36] Y. Zhao, Y. Liu, X. Liu, M. Sun, F. Qi, and Y. Zheng, "Self-adapting spiking neural p systems with refractory period and propagation delay," *Information Sciences*, vol. 589, pp. 80–93, 2022.
- [37] H. M. Lehmann, J. Hille, C. Grassmann, and V. Issakov, "Leaky integrate-and-fire neuron with a refractory period mechanism for invariant spikes," in *2022 17th Conference on Ph.D Research in Microelectronics and Electronics (PRIME)*, 2022, pp. 365–368.
- [38] A. Gupta, A. Gu, and J. Berant, "Diagonal state spaces are as effective as structured state spaces," in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, 2022.
- [39] S. Ioffe and C. Szegedy, "Batch normalization: accelerating deep network training by reducing internal covariate shift," in *Proceeding. of the International Conference on Machine Learning (ICML)*, vol. 37, 2015, p. 448–456.
- [40] M. Yao *et al.*, "Spike-driven transformer," *Advances in neural information processing systems*, vol. 36, pp. 64 043–64 058, 2023.
- [41] Y. LeCun, "The mnist database of handwritten digits," 2017. [Online]. Available: <http://yann.lecun.com/exdb/mnist/>
- [42] W. Dai, C. Dai, S. Qu, J. Li, and S. Das, "Very deep convolutional neural networks for raw waveforms," in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017, pp. 421–425.
- [43] J. Kaiser, H. Mostafa, and E. Neftci, "Synaptic plasticity dynamics for deep continuous local learning (DECOLLE)," *Frontiers in Neuroscience*, vol. Volume 14 - 2020, 2020.
- [44] H. Zheng, Y. Wu, L. Deng, Y. Hu, and G. Li, "Going deeper with directly-trained larger spiking neural networks," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 12, pp. 11 062–11 070, May 2021.

- [45] F. Moro, P. V. Aceituno, L. Kriener, and M. Payvand, “The role of temporal hierarchy in spiking neural networks,” *arXiv preprint arXiv:2407.18838*, 2024.
- [46] A. Shaban, S. S. Bezugam, and M. Suri, “An adaptive threshold neuron for recurrent spiking neural networks with nanodevice hardware implementation,” *Nature Communications*, vol. 12, no. 1, p. 4234, 2021.
- [47] P. A. Merolla *et al.*, “A million spiking-neuron integrated circuit with a scalable communication network and interface,” *Science*, vol. 345, no. 6197, pp. 668–673, 2014.
- [48] K. M. Stewart, T. Shea, N. Pacik-Nelson, E. Gallo, and A. Danielescu, “Speech2spikes: Efficient audio encoding pipeline for real-time neuromorphic systems,” in *Proc. of the Annual Neuro-Inspired Computational Elements Conf. (NICE)*, 2023, p. 71–78.
- [49] G. Lenz, K. Chaney, S. B. Shrestha, O. Oubari, S. Picaud, and G. Zarrella, “Tonic: event-based datasets and transformations.” 2021.
- [50] A. Subramoney, K. K. Nazeer, M. Schöne, C. Mayr, and D. Kappel, “Efficient recurrent architectures through activity sparsity and sparse back-propagation through time,” 2023.
- [51] S. U. Innocenti, F. Becattini, F. Pernici, and A. Del Bimbo, “Temporal binary representation for event-based action recognition,” in *2020 25th International Conference on Pattern Recognition (ICPR)*. IEEE, 2021, pp. 10 426–10 432.